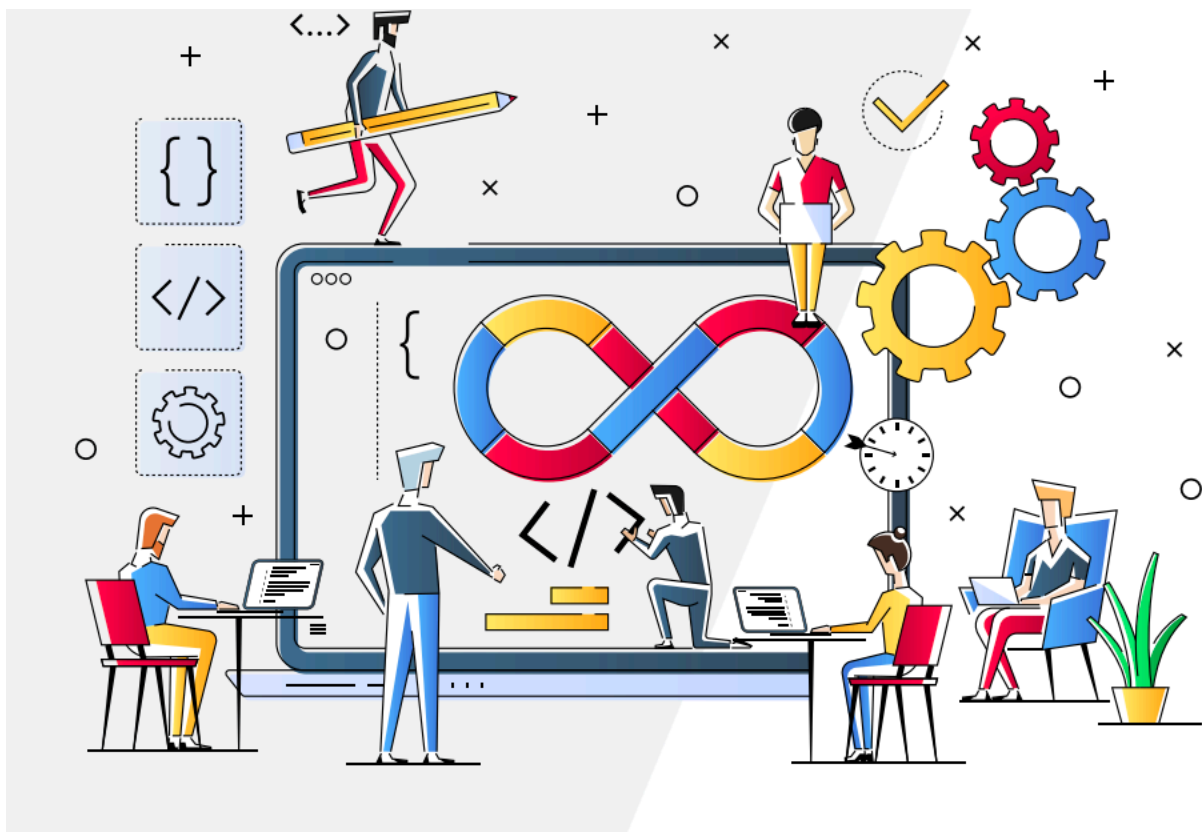


#ContecTechTeam



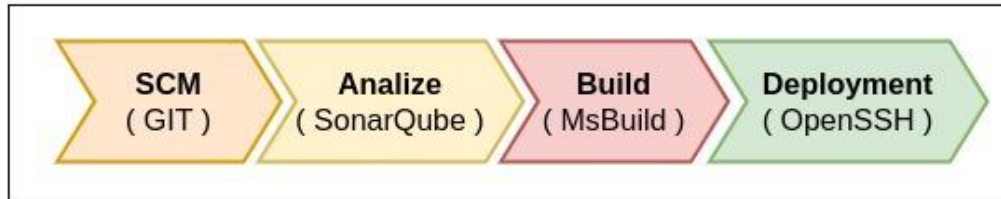
CONTINUOUS INTEGRATION & DELIVERY (CI/CD) PIPELINE

Version 1.2



Contents	Page
1- Stages in a CI/CD pipeline	3
2- CI/CD pipeline	3
3- Install OpenSSH On Windows	4
4- Jenkins installation in windows	6
5- CI/CD pipeline using Jenkins (Dev).....	8
6- CI/CD pipeline (QA)	11
7- CI/CD pipeline (production)	13
8- How To Backup Jenkins Data and Configurations	14
9- SonarQube Introduction	16
10- Install SonarQube on Windows	17
11- How to integrate sonarqube with jenkins	17
12- Create a project in sonarqube	22
13- Quality Profile in sonarqube	25
14- Associate Quality profile with a project	25
15- SonarQube Webhooks	26
16- SonarQube Quality Gate	27

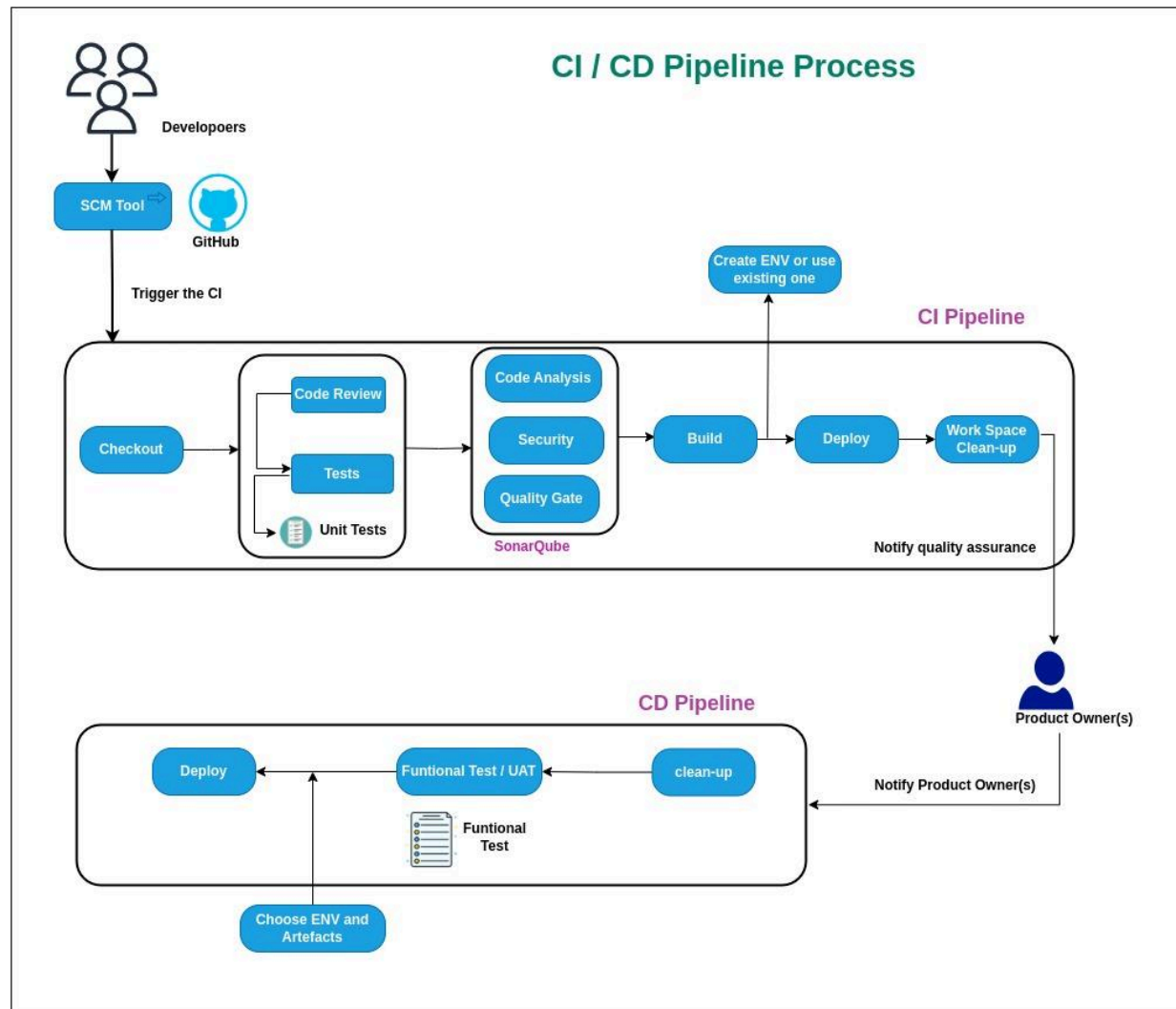
1- Stages in a CI/CD pipeline



2- CI/CD pipeline

Pipeline represents a simple application development process.

1. The developer develops the code and commits the changes to a centralized GitHub repository.
2. Jenkins gets the new code and carries out the automated analysis of code through sonarqube , if passed the quality gates of sonarqube, build the project.
3. The final package is uploaded to Dev / QA / Production via the 'publish over ssh' plugin of jenkins.



Note: if the remote server is a Windows server, OpenSSH must be installed on the remote server for deployment through 'publish over ssh'.

3- Install OpenSSH On Windows

3.1- on Windows Server 2016

<https://creodias.docs.cloudferro.com/en/latest/networking/How-to-install-OpenSSH-on-Windows-Server-2016-VM-on-Creodias.html>

- 1- Log in to your Remote Server via RDP
- 2- Download the OpenSSH (OpenSSH-Win64.zip)server from GitHub (<https://github.com/PowerShell/Win32-OpenSSH/releases>)
- 3- Open the downloaded file and copy the "OpenSSH-Win64" folder to "C:Program Files".

Open the downloaded file and copy the "OpenSSH-Win64" folder to "C:\Program Files".

4- Go to "C:\Program Files\OpenSSH-Win64" and edit the "sshd_config_default" file.

5- Find and uncomment following lines by removing the "#" at the beginning then save the file.

Port 22

PasswordAuthentication yes

6- Now you have to modify the system environment variable. To do it run Windows PowerShell as administrator

type the following command to set Environment variable

```
setx PATH "$env:path;C:\Program Files\OpenSSH-Win64" -m
```

7- run the install script.

```
cd "C:\Program Files\OpenSSH-Win64"; .\install-sshd.ps1
```

8- Enable automatic startup and start OpenSSH from services

9- Connect to Remote via putty OR GIT Bash

```
ssh <userName>@<your-vm-ip-address>
```

3.2- On Windows 11:

https://winscp.net/eng/docs/guide_windows_openssh_server

- On Windows 11:
 - Go to *Settings > Apps > Optional features* and click on *View features*.
 - Locate the "OpenSSH server" feature, select it, click *Next*, and then click *Install*.
- On Windows 10 (version 1803 and newer):
 - Go to *Settings > Apps > Apps & features > Optional features* and click on *Add a feature*.
 - Locate the "OpenSSH server" feature, expand it, and select *Install*.

4- Jenkins installation in windows

Prerequisites

Java Development Kit (JDK) 17 or Higher: Ensure that Java Development Kit (JDK) version 17 or higher is installed on the Windows system. Jenkins requires Java to run.

3.1- Downloading and Installing Java JDK

- a) Download Oracle Java 17, 64-bit from [Oracle's website](#).
- b) Run the downloaded installer and follow the on-screen instructions to complete the Java installation.

3.2- Setting up Environment Variables for Java in Windows

- a) Navigate to your system environment variables. You can search for "Edit the system environment variables" in the Start menu.
- b) Click on "Environment Variables."
- c) Under "System variables," find the variable named "Path" and click "Edit."
- d) Add the path to your Java installation directory (e.g., C:\Program Files\Java\jdk-17.0.5\bin) to the end of the variable value, separated by a semicolon (;).
- e) Click "OK" to save the changes.

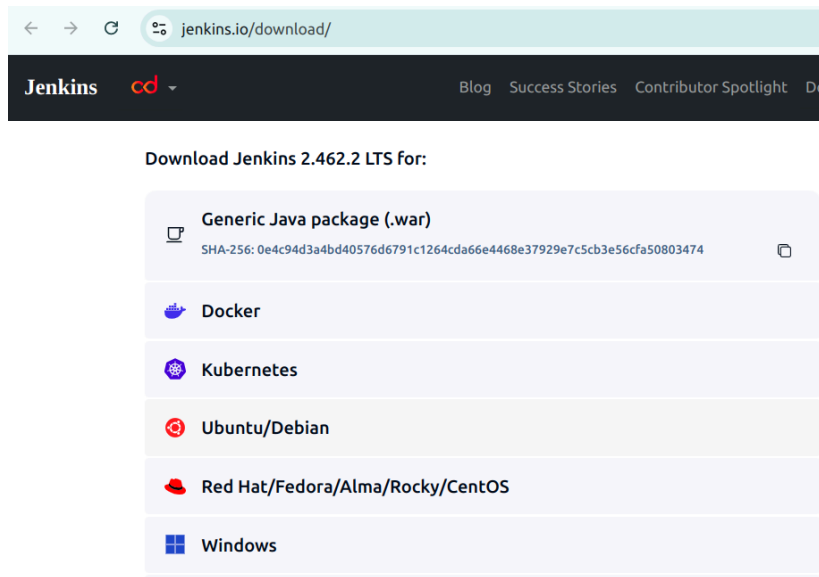
3.3-Checking Installed Java Version

Before proceeding, verify the installed Java version:

- a) Open Command Prompt (Win + R, type cmd, and press Enter).
- b) Type java -version and press Enter.
- c) The output should display "java version 17.0.5" (or a higher version) to confirm compatibility.

3.4- Downloading Jenkins LTS

- a) Download the latest Long-Term Support (LTS) version of Jenkins from the official website: <https://www.jenkins.io/download/>

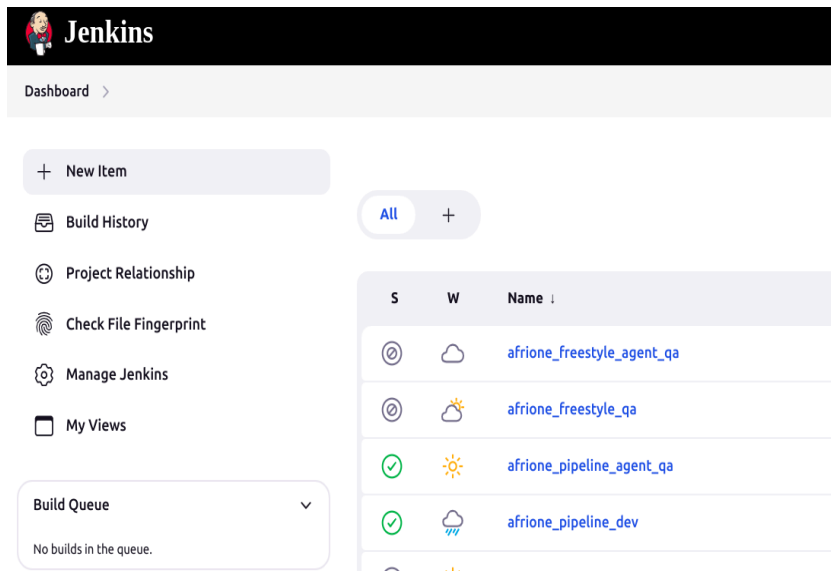


- b) Run the downloaded Jenkins Windows installer (.msi file).
- c) Choose the installation type (Windows service recommended).
- d) Specify the port number for Jenkins (default is 8080).
- e) Choose the Java directory (should point to your previously installed JDK).
- f) Choose the installation location on your local hard drive.
- g) Click "Install" to begin the installation process.
- h) Upon successful installation, you'll be provided with the initial admin password. Copy this password for later use.
- i) In your web browser, navigate to <http://localhost:8080> (or the port you used during installation) to access the Jenkins dashboard.

5- CI/CD pipeline using Jenkins (Dev)

Step 1: Opening Jenkins

Login to Jenkins and click on "New Item."



Step 2: Naming the pipeline

Select the "Pipeline" option from the menu, provide a name for the pipeline, and click "OK."

Enter an item name

my-test-pipeline

» Required field

Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps, archives artifacts and sending email notifications.

Pipeline
Orchestrates long-running activities that can span multiple build agents. Suitable for organizing complex activities that do not easily fit in free-style job type.

Multi-configuration project
Suitable for projects that need a large number of different configurations, such as builds, etc.

Folder
Creates a container that stores nested items in it. Useful for grouping things together in a separate namespace, so you can have multiple things of the same name.

OK

Step 3: Configuring the pipeline

The most important section is the "Pipeline Definition" section, where you can define the stages of the pipeline. Pipeline supports both declarative and scripted syntaxes

Pipeline

Definition

Pipeline script

Script ?

```

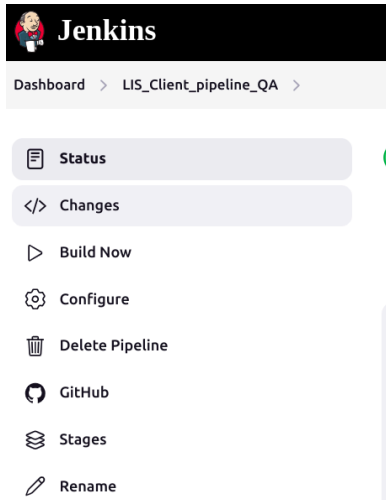
1 pipeline {
2   agent any
3
4   stages {
5
6     stage('Checkout Scm') {
7       steps {
8         // Workspace Cleanup Plugin, clean workspace before build as follows */
9         cleanWs() // clean work space
10
11         git branch: 'QA', url: 'https://ghp_mNek0JEAIUVBjMQlRfLMSnhtIKnLVtixlpi8github.com/connectechrepo/rp_consumer_web.git'
12         //git 'https://ghp_mNek0JEAIUVBjMQlRfLMSnhtIKnLVtixlpi8github.com/connectechrepo/rp_consumer_web.git'
13       }
14     }
15
16     stage('Build') {
17       steps {
18
19         echo 'Building .....'
20
21         //build 'LIS_Client_QA'
22
23         script {
24           bat '''
25             "C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\MSBuild\Current\Bin\MSBuild.exe" C:/ProgramData/Jenkins
26           '''
27         }
28       }
29     }
30   }
31 }
32
33
34

```

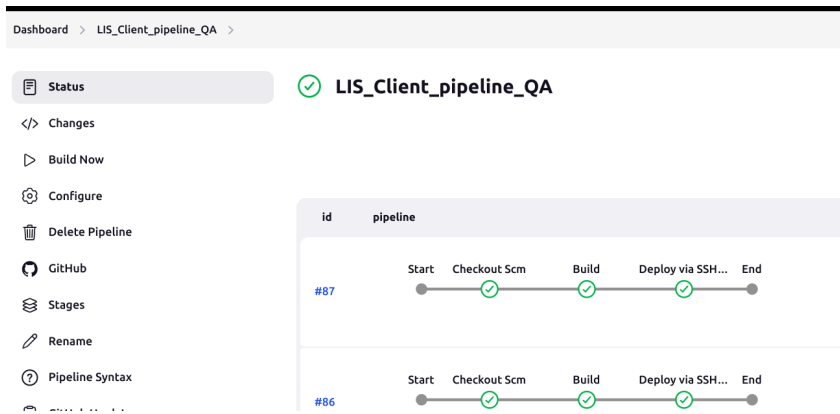
Save Apply

Step 4: Executing the pipeline

Click on "Build Now" to execute the pipeline.



This will result in the pipeline stages getting executed and the result getting displayed in the "Stage View" section. We've only configured a single pipeline stage, as indicated here:



We can verify that the pipeline has been successfully executed by checking the console output for the build process.

Dashboard > LIS_Client_pipeline_QA > #87

- Status
- Changes
- Console Output**
- View as plain text
- Edit Build Information
- Delete build '#87'
- Polling Log
- Timings
- Git Build Data
- Pipeline Overview
- Pipeline Console
- Restart from Stage

 **Console Output**

```

Started by GitHub push by contextechrepo
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\jenkins\workspace\LIS_Client_pipeline_QA
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Scm)
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] done
[Pipeline] git
The recommended git tool is: C:\Program Files\Git\cmd\git.exe
No credentials specified
Cloning the remote Git repository
Cloning repository https://ghp_mNek0JEAIUYBjM0lrFLMSmhtIKmLVt1xLIp1@github.com/contextechrepo
> C:\Program Files\Git\cmd\git.exe init C:\ProgramData\Jenkins\jenkins\workspace\LIS_Client_pipeline_QA
Fetching upstream changes from https://ghp_mNek0JEAIUYBjM0lrFLMSmhtIKmLVt1xLIp1@github.com:contextechrepo
> C:\Program Files\Git\cmd\git.exe fetch --tags --force --progress --

```

6- CI/CD pipeline (QA) (Deploy code in Remote Server through jenkins)

Note : All the stages will same for QA, build will deploy in remote server so we will use OpenSSH on remote window server and 'publish over ssh' plugin in jenkins

In the Deploy stage of the pipeline will use the 'Publish Over SSH' jenkins plugin and install OpenSSH in a remote window machine.

5.1- Install jenkins plugin

Dashboard --> Manage Jenkins --> Plugins → Available plugins
Search Publish Over SSH select and install

Dashboard > Manage Jenkins > Plugins

Plugins

Search: Publish Over SSH

- Updates 28
- Available plugins
- Installed plugins**
- Advanced settings

Name

[Publish Over SSH 1.25](#)

Send build artifacts over SSH

[Report an issue with this plugin](#)

5.2- Add Global Configuration

Dashboard → Manage Jenkins → System
Add SSH Server with it's credentials

SSH Server

Name ?

ssh-lis-prod

Hostname ?

10.1.32.21

Username ?

Administrator

Remote Directory ?

/

☐ Avoid sending files that have not changed ?

Advanced ▾

✎ Edited

Test Configuration

Save

Apply

5.3- Use the above configuration in pipeline as below in deployment stage

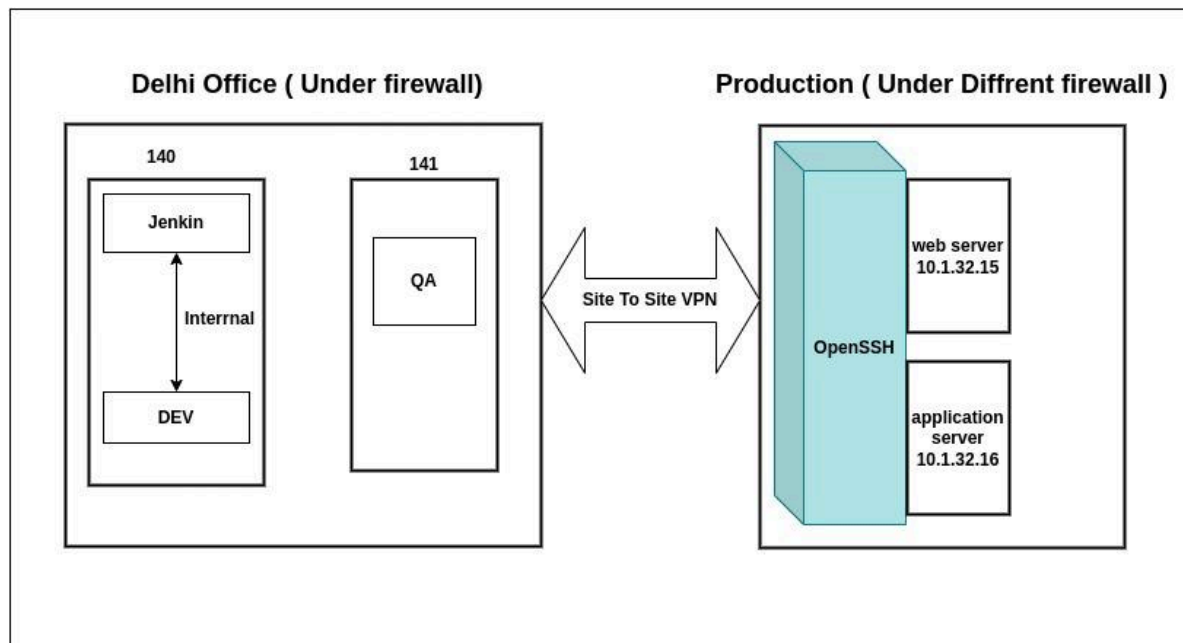
```
stage('Deploy via SSH Transfer') {
  steps([$class: 'BapSshPromotionPublisherPlugin']) {
    ws("E:\\\\LIS_Publish_qa\\") {
      echo "work space has changed "
      sshPublisher(
        continueOnError: false, failOnError: true,
        publishers: [
          sshPublisherDesc(
            configName: "sshqa141", // Connect to 141 server through ssh
            verbose: true,
            transfers: [
```

```

sshTransfer(
  remoteDirectory: "D:\\\\LIS_Publish_qa",
  sourceFiles: "**/*"
),
//sshTransfer(execCommand: "pwd"),
//sshTransfer(sourceFiles: "helm/**",)
]
)
]
)
}
}
}

```

7- CI/CD pipeline (production)

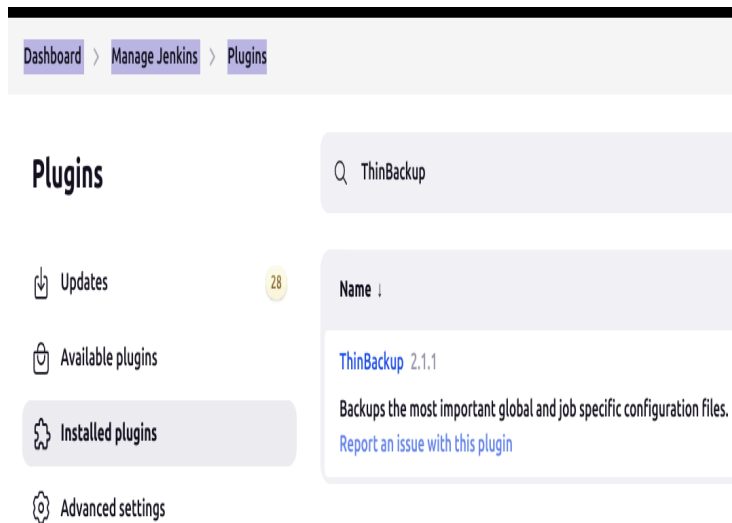


8- How To Backup Jenkins Data and Configurations

We will use the 'ThinBackup' plugin of Jenkins. It includes, job configs, build logs, plugins, plugin configuration etc.

7.1- Install the plugin “ThinBackup” as shown below:

Dashboard --> Manage Jenkins --> Plugins --> Available plugins



7.2- ThinBackup Configuration

Dashboard --> Manage Jenkins --> System --> ThinBackup Configuration

Enter the backup options as shown below :

Backup directory ?

C:\jenkins_backup

Backup schedule for full backups ?

Backup schedule for differential backups ?

Max number of backup sets ?

-1

Files excluded from backup (regular expression) ?

☒ **Wait until Jenkins is idle to perform a backup** ?

Force Jenkins to quiet mode after specified minutes ?

120

☐ **Backup build results** ?

☒ **Backup userContent folder** ?

☐ **Backup next build number file** ?

☒ **Backup plugins archives** ?

☐ **Backup config-history folder** ?

☒ **Backup additional files** ?

Files included in backup (regular expression)

☐ **Clean up differential backups** ?

☒ **Move old backups to ZIP files** ?

Save **Apply**

7.3- Usage of thinBackup plugin of jenkins

A- Backup Now

Triggers a manual full backup right now. No restart required.

Dashboard --> Manage Jenkins --> ThinBackup


Jenkins

Dashboard > Manage Jenkins > ThinBackup

+ New Item

 Build History

 Project Relationship

 Check File Fingerprint

 Manage Jenkins

 My Views

ThinBackup

Settings are now integrated in global configuration.

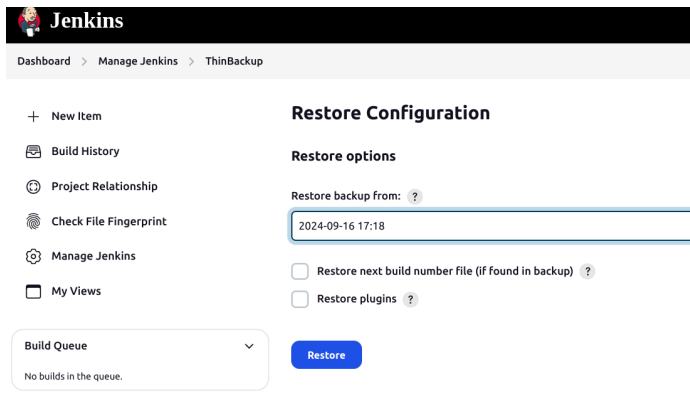



B- Restore

Select the date of the backup you would like to restore.

After the restore finishes you are redirected to the plugin main page.

To activate the restored settings you need to restart Jenkins.



9- SonarQube Introduction

8.1-A Jenkins Pipeline job is started.

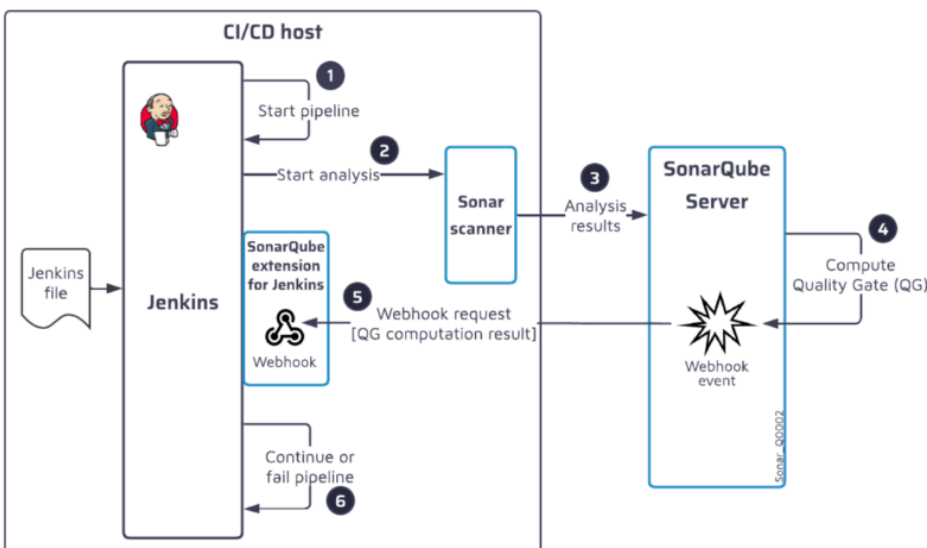
8.2- The job triggers the analysis by the SonarScanner.

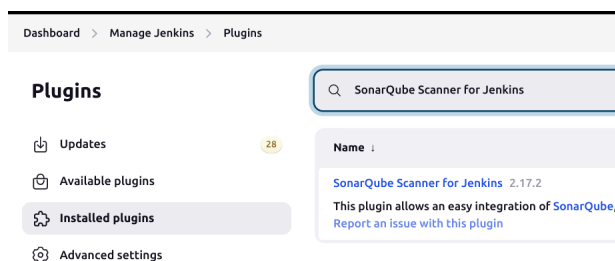
8.3- The SonarScanner sends the results to the SonarQube Server.

8.4- SonarQube Server completes the analysis, computes quality gate configured for the project, and checks if the project fails or passes quality gate.

8.5- The SonarQube Server sends the pass or failure result back to the Jenkins webhook exposed by the SonarQube extension for Jenkins.

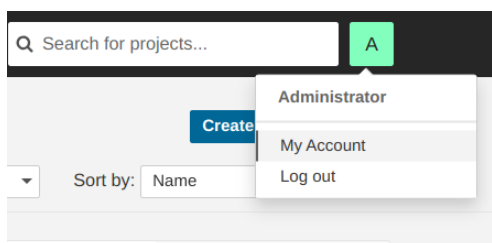
8.6- The pipeline job continues (in case of a pass) or fails (otherwise).



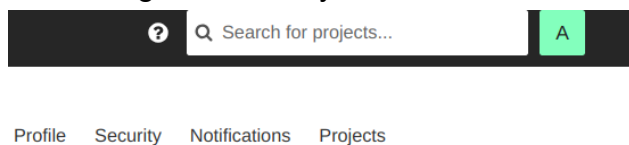


Step 2: Generate Tokens for jenkins from sonarqube

- 1- Now go to sonarqube server (<http://192.168.0.140:9000/>) to generate secret from sonarqube , which will be added to jenkins as credentials
- 2- go to extreme right corner → userName → My Account



3- go to 'security'



4- Generate Tokens

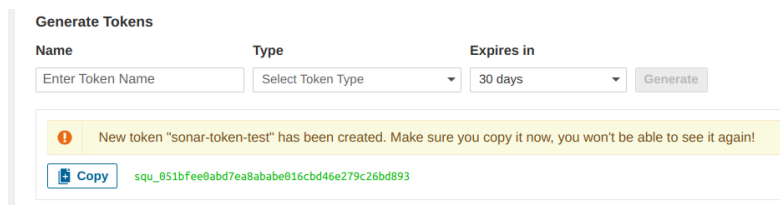
Tokens

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password

Generate Tokens

Name	Type	Expires in	
sonar-token	User Token	No expiration	<button>Generate</button>

5- Copy the generated token and save it.



Step 3: add credential inside jenkins , token that is generated in step 2.

Dashboard → Manage Jenkins → Credentials → System Global credentials (unrestricted)

New credentials

Kind

Secret text

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

ID ?

sonar-token

1 This ID is already in use

Description ?

sonar-token

Create

Select 'Kind' as Secret text

Step 4: add sonarqube server inside jenkins

Go to Dashboard → Manage Jenkins → System

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment

☒ Environment variables

SonarQube installations

List of SonarQube installations

Name

sonar-server

Server URL

Default is http://localhost:9000

http://192.168.0.140:9000/

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonar-token

+ Add

Advanced ▾

Add SonarQube

Save **Apply**

Note : Make sure to checkbox must be checked Environment variables
Server authentication token will appear in dropdown which is added in global credentials.

Step 5: setup a sonar scanner which will scan the code.

Dashboard --> Manage Jenkins --> Tools

SonarScanner for MSBuild installations

SonarScanner for MSBuild installations ^  Edited

Add **SonarScanner** for MSBuild

≡ **SonarScanner** for MSBuild

Name

dotnet-sonar-scanner

☒ Install automatically ?

≡ Install from GitHub

Version

SonarScanner for MSBuild 5.15.1.88158 - .NET Fwk 4.6

Add Installer ▾

Add **SonarScanner** for MSBuild

Save

Apply

Step 6: Write the pipeline, to scan github repositories for code scanning.

```

pipeline {
  agent any
  stages {
    stage('Checkout Scm') {
      steps {
        cleanWs() // clean work space
        git
        'https://ghp_mNek0mIVt1xllp1@github.com/contectechrepo/rp_consumer_web.git'
      }
    }

    stage('SonarScanner') {
      steps {
        withSonarQubeEnv('sonar-server') { // If you have configured more than one
        global server connection, you can specify its name
          bat "
C:\\ProgramData\\Jenkins\\jenkins\\tools\\hudson.plugins.sonar.SonarRunnerInstallation\\sonner-scanner\\bin\\sonar-scanner.bat -D"sonar.projectKey=lis_client_dev"
-D"sonar.sources=." -D"sonar.host.url=http://192.168.0.140:9000"
-D"sonar.login=sqp_759a0bc48f15ee2f"

          "
        }
      }
    }

    stage("SonarQualityGate"){
      steps {
        script {
          timeout(time: 1, unit: 'HOURS') {
            def qg = waitForQualityGate()
            if (qg.status != 'OK') {
              error "Pipeline aborted due to quality gate failure: ${qg.status}"
            }
          }
        }
      }
    }
  }
}

```

```

    }
  }
}

stage('Build') {
  steps {
    script {
      bat "
        \"C:\\Program Files (x86)\\Microsoft Visual
Studio\\2022\\BuildTools\\MSBuild\\Current\\Bin\\MSBuild.exe\"
C:/ProgramData/Jenkins/.jenkins/workspace/LIS_Client_pipeline_Dev/LIS.sln
/p:DeployOnBuild=true /p:WebPublishMethod=FileSystem /p:Configuration=Release
/p:OutputPath=\"E:/LIS Publish\"

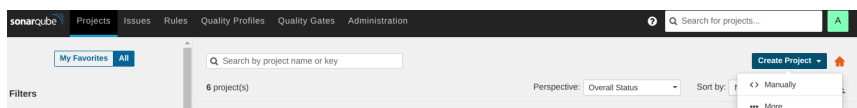
        \"
        // bat \"
        //
\"C:\\ProgramData\\Jenkins\\.jenkins\\tools\\hudson.plugins.msbuild.MsBuildInstallati
on\\Jenkins_MSBuild\\vs_BuildTools.exe\"
C:/ProgramData/Jenkins/.jenkins/workspace/LIS_Client_pipeline_Dev/LIS.sln
/p:DeployOnBuild=true /p:WebPublishMethod=FileSystem /p:Configuration=Release
/p:OutputPath=\"E:/LIS Publish\"
        // \"
      }
    }
  }
}
}
}

```

12- Create a project in sonarqube

11.1- open sonarqube <http://192.168.0.140:9000/>

11.2- Go to projects → create project → manually



11.3- fill project detail

Create a project

All fields marked with * are required

Project display name *

 ✓
Up to 255 characters. Some scanners might override the value you provide.

Project key *

 ✓
The project key is a unique identifier for your project. It may contain 400 characters. Allowed characters are alphanumeric, '-' (dash), '_' (underscore), '.' (period) and ':' (colon), with at least one non-digit.

Main branch name *

 ✓
The name of your project's default branch. [Learn More](#)

[Set Up](#)

11.4- Click on Locally

afrione-v2

☆

dev

•

[Overview](#)
[Issues](#)
[Security Hotspots](#)
[Measures](#)
[Code](#)

How do you want to analyze your repository?

Do you want to integrate with your favorite CI? Choose one of the



With Jenkins



With GitHub Actions

Are you just testing or have an advanced use-case? Analyze yo



Locally

11.5- Generate a project token

afrione-v2

☆

dev

•

[Overview](#)
[Issues](#)
[Security Hotspots](#)
[Measures](#)
[Code](#)
[Activity](#)

Analyze your project

We initialized your project on SonarQube, now it's up to you to launch analyses!

1

Provide a token

☒ **Generate a project token**

Token name ⓘ

Expires in

No expira... ▾

[Generate](#)

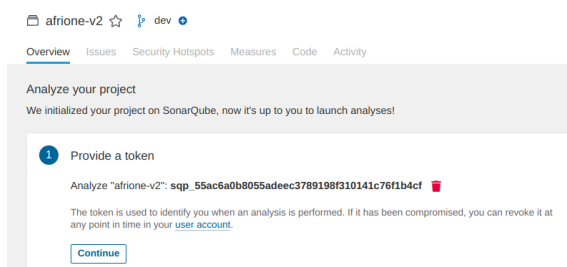
ⓘ

Please note that this token will only allow you to analyze the current project. To use the same token to analyze multiple projects, you need to generate a token for your user account. See the [documentation](#) for more information.

☐ Use existing token

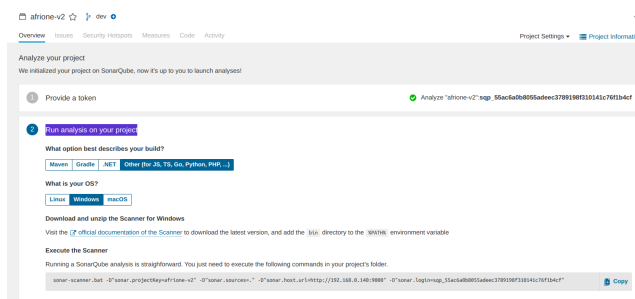
The token is used to identify you when an analysis is performed. If it has been created at any point in time in your [user account](#).

Save the generated token and Continue



11.6- to run analysis on your project

Select language and OS will provide the command to analyze, which will use in pipeline as below



Add below stage in pipeline

```
stage('SonarScanner') {
    steps {
        withSonarQubeEnv('sonar-server') { // If you have configured more than
            one global server connection, you can specify its name
            bat '''
C:\\ProgramData\\Jenkins\\.jenkins\\tools\\hudson.plugins.sonar.SonarRunnerInstallation\\sonar-scanner\\bin\\sonar-scanner.bat -D"sonar.projectKey=lis_client_dev"
-D"sonar.sources=." -D"sonar.host.url=http://192.168.0.140:9000"
-D"sonar.login=sqp_4ab85a0bc48f15ee2f"

'''
        }
    }
}
```

13- Quality Profile in sonarqube : define rules for analyzing code. When a project is analyzed, SonarQube uses the active quality profile

12.1- Create a Quality Profile

Quality Profile → Create → Extend an existing quality profile

New Quality Profile

What type of profile do you want to create?

Extend an existing quality profile
Create a child quality profile inheriting its parent's active rules. Changes to the parent profile will impact the child profile.

Copy an existing quality profile
Create a new quality profile as a replica of the copied quality profile. The two profiles will then evolve independently.

Create a blank quality profile
Create a new quality profile with no active rules by default.

All fields marked with * are required

Language *
PHP ✓

Profile to extend *
Sonar way (Built-in) ✓

Name *
contec-php ✓

14- Associate Quality Profile with a project

13.1- Projects → projectName → project Settings → Quality Profile

afrione-v2 ☆ dev

Overview Issues Security Hotspots Measures Code Activity

Project Settings ▾

- General Settings
- New Code
- Import / Export
- Quality Profiles
- Quality Gate

How do you want to analyze your repository?

Do you want to integrate with your favorite CI? Choose one of the following tutorials.

13.2- Add Language

afrione-v2 ☆ dev

Overview Issues Security Hotspots Measures Code Activity

Project Settings ▾

Quality Profiles ⓘ

Manage project Quality Profiles

Choose which profile is associated with this project on a language-by-language basis.

Add a new language

Manually configure a specific profile for a new language before the next analysis.

[Add language](#)

13.3- Select Quality Profile

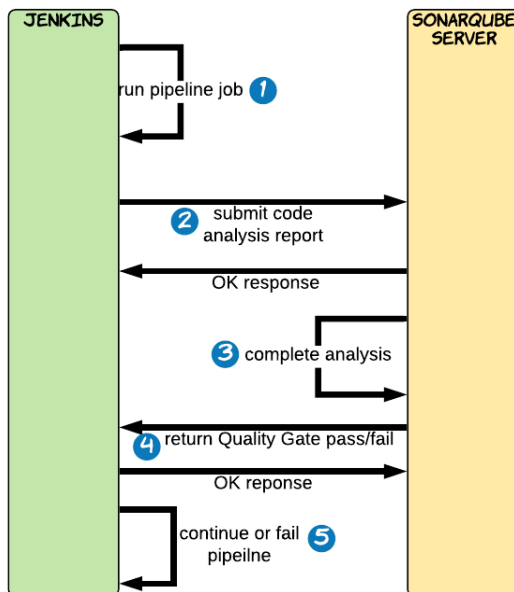
Add a language

Choose a language
 PHP

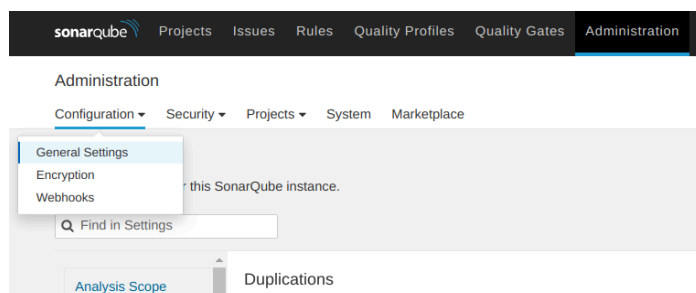
Choose a profile
 pontec-php

Save Cancel

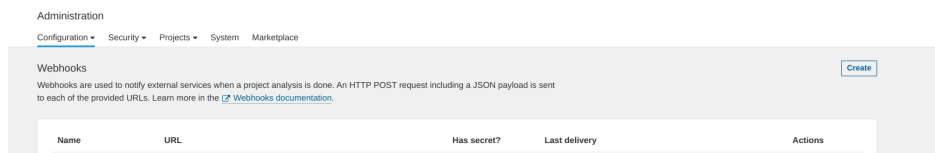
15- SonarQube Webhooks : To notify jenkins that analysis is completed.



14.1- Create Webhooks to notify jenkins that analysis is completed
 Administration → Configuration → Webhooks



Click on Create



Fill the form

Name : any unique name

URL : <http://192.168.0.140:8080/sonarqube-webhook/> (host-url/sonarqube-webhook)

Secret : optional, not mandatory

Create Webhook

All fields marked with * are required

Name *
new-jenkin-webhook ✓

URL *
http://192.168.0.140:8080/sonarqube-webhook/ ✓

Server endpoint that will receive the webhook payload, for example:
"http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example:
"https://myLogin:myPassword@my_server/foo"

Secret
[Empty field]

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

Create Cancel

16- SonarQube Quality Gate : quality gate to force the build to fail if some conditions are not met during code analysis.

Adding a quality gate

a) Login to SonarQube, Click on Quality gate → Create, enter some name

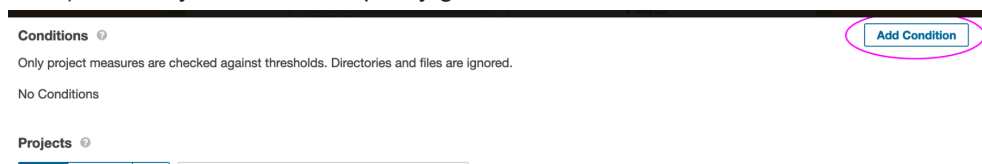
Create Quality Gate

All fields marked with * are required

Name *
contecQualityGate

Save Cancel

a) Once you create the quality gate. Click on Add condition.



b) Select new bugs from the drop down and enter 1 as error

d) Choose your Web App, by clicking on App. and select a project.

e) Now to go Jenkins, add new stage in a pipeline job:

```
stage("Quality Gate") {
    timeout(time: 1, unit: 'HOURS') {
        waitForQualityGate abortPipeline: true
    }
}
```

Now you should see the Jenkins console output like this:

```
[[1;34mINFO[m] Finished at: 2021-01-15T19:57:50Z
[[1;34mINFO[m] [1m-----
[Pipeline] }
[Pipeline] // withSonarQubeEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Quality Gate)
[Pipeline] timeout
Timeout set to expire in 1 hr 0 min
[Pipeline] {
[Pipeline] waitForQualityGate
Checking status of SonarQube task 'AXcHoCGA21DPK87k-ceg' on server 'SonarQube'
SonarQube task 'AXcHoCGA21DPK87k-ceg' status is 'SUCCESS'
SonarQube task 'AXcHoCGA21DPK87k-ceg' completed. Quality gate is 'ERROR'
[Pipeline] }
[Pipeline] // timeout
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: Pipeline aborted due to quality gate failure: ERROR
Finished: FAILURE
```